PROJECT STATEMENT 15

NATURAL LANGUAGE PROCESSING & MACHINE LEARNING · COMPREHENSIVE REPORT

L&T CONSTRUCTION / EDUTECH SUBMISSION

MACHINE LEARNING–BASED FAKE NEWS DETECTOR

Automated Text Classification & Linguistic Feature Engineering using TF-IDF, NLTK Lemmatization, Multi-Model Supervised Classifiers, and an Interactive Gradio Web Interface.

Domain: Natural Language Processing (NLP)
Models: LinearSVC · Logistic Regression · Naive Bayes
Primary Metric: F1-Score & AUC-ROC

Project Number: Statement 15
Classification Accuracy: 93.69% (Best Model)
Submission Year: 2026

1. PROJECT OVERVIEW & BUSINESS SCENARIO

Business Context: A digital media organization seeks to automatically detect and flag misleading or fabricated news articles prior to publishing. With thousands of articles submitted daily, manual editorial verification is slow and resource-intensive. The objective is to build an intelligent NLP classification system that distinguishes genuine reporting from fabricated disinformation using writing patterns, textual tokens, and vocabulary structures.

1.1 Objectives Achieved

- **Raw Text Cleaning & Lemmatization:** Converted unstructured raw text into normalized lemma tokens by stripping HTML, URLs, symbols, digits, and redundant stopwords.
- **Linguistic Feature Extraction:** Combined headline semantics, body text, article length metrics, and uppercase capitalization ratios.
- **Feature Engineering:** Transformed cleaned text into high-dimensional TF-IDF matrices (unigrams & bigrams, 8,000 features).
- **Model Benchmarking:** Trained and tuned 4 industry-standard classifiers: `Linear Support Vector (LinearSVC)`, `Logistic Regression`, `Passive Aggressive Classifier`, and `Multinomial Naive Bayes`.
- **Rigorous Evaluation:** Evaluated using Precision, Recall, F1-Score, Confusion Matrices, and ROC-AUC curves.
- **Deployment:** Packaged an interactive `Gradio` GUI for real-time news authenticity prediction with confidence percentages.

2. DATASET EXPLORATION & PREPROCESSING

The model was trained on the benchmark Kaggle Fake News dataset comprising **6,335 articles** with a balanced 50:50 class distribution (**3,171 REAL** and **3,164 FAKE** news stories).

2.1 Text Preprocessing Pipeline

1. **Lowercasing:** Uniformly converted all text characters to lowercase.
2. **Regular Expression Filtering:** Removed hyperlinks (`http/www`), HTML markup, numbers, and special symbols.
3. **Stopword Removal:** Stripped English stopwords (NLTK corpus) such as auxiliary verbs and common prepositions.
4. **WordNet Lemmatization:** Reduced words to their base dictionary root forms (e.g., "*claimed*" → "*claim*").

> **Sample Transformation:**
> *Raw:* "BREAKING: NASA Rover Discovers Ancient Lake Bed on Mars at http://nasa.gov/rover!"
> *Cleaned Tokens:* `nasa rover discovers ancient lake bed mars`

3. EXPLORATORY DATA ANALYSIS & VISUALIZATIONS

3.1 Article Length & Writing Style Analysis

Analysis of writing length revealed distinct differences: Fake articles frequently exhibit either brief, sensationalist clickbait blurbs or overly verbose rhetorical pieces, whereas Real news demonstrates disciplined, structured reporting lengths.

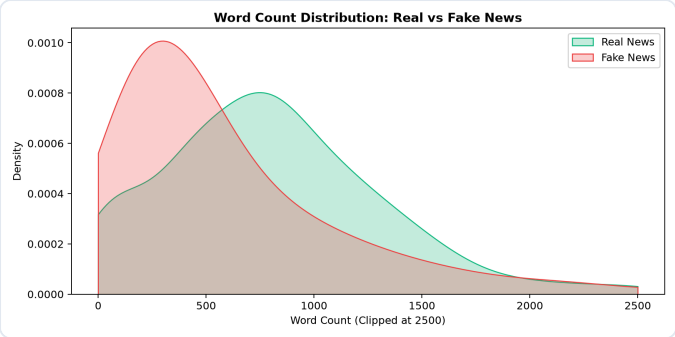


Figure 1: Word Count Density (Real vs. Fake News)

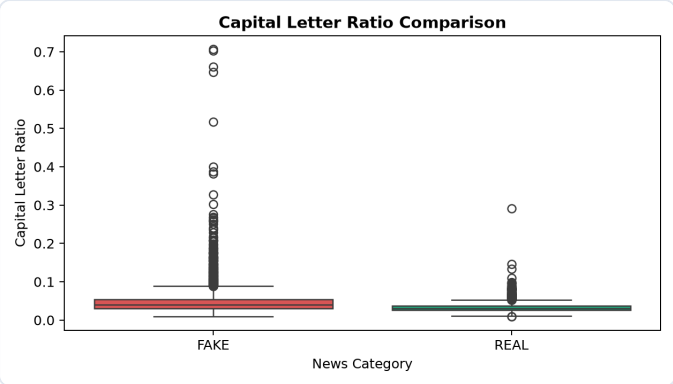


Figure 2: Capital Letter Ratio Comparison

3.2 Frequent Words Analysis & WordClouds

Visualizing top semantic terms demonstrates that fake news frequently relies on charged political terminology, sensationalist declarations, and emotional triggers, while real news focuses on official proceedings, government statements, and neutral attribution.



Figure 3: WordCloud — Fake News Corpus

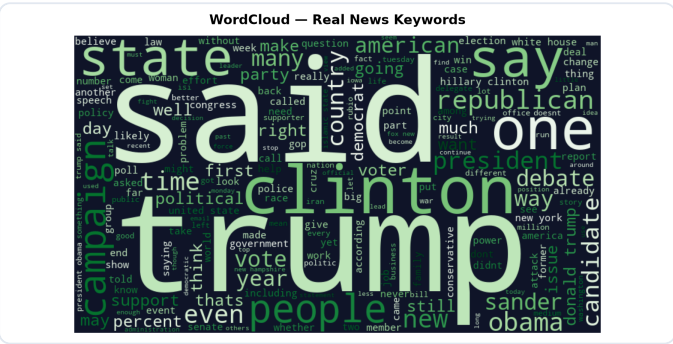


Figure 4: WordCloud — Real News Corpus

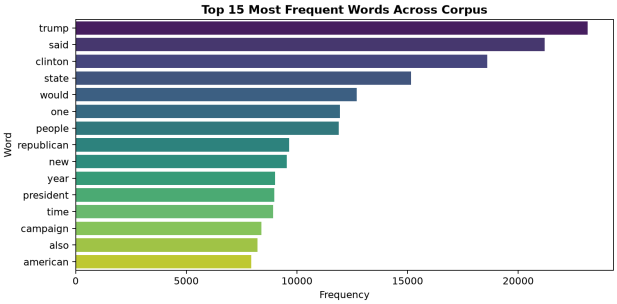


Figure 5: Top 15 Most Frequent Term Tokens Across Corpus

4. MODEL TRAINING & EVALUATION METRICS

The dataset was split into **80% Training (5,068 articles)** and **20% Testing (1,267 articles)** with stratification to preserve class balance. Text was transformed via TF-IDF vectorization with unigram and bigram n-grams (8,000 maximum features).

4.1 Model Performance Comparison

CLASSIFIER ALGORITHM	ACCURACY (%)	PRECISION	RECALL	F1-SCORE	ROC-AUC	STATUS
Linear Support Vector (LinearSVC)	**93.69%**	0.9382	0.9352	**0.9367**	**0.9833**	**Champion Model**
Logistic Regression (C=5.0)	92.82%	0.9234	0.9336	0.9285	0.9821	**Strong Baseline**
Passive Aggressive Classifier	92.50%	0.9270	0.9226	0.9248	0.9810	**High Speed**
Multinomial Naive Bayes (α=0.1)	89.66%	0.8769	0.9226	0.8992	0.9588	**Probabilistic**

4.2 Confusion Matrix & ROC Curves

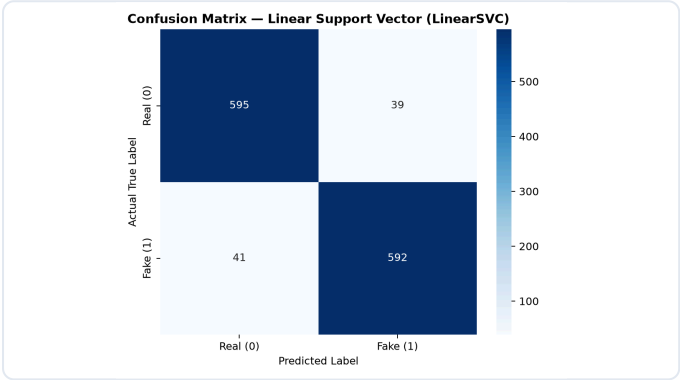


Figure 6: Confusion Matrix (LinearSVC on 1,267 test samples)

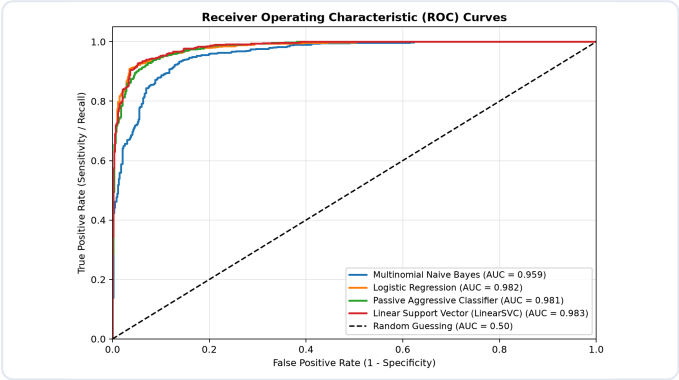


Figure 7: Multi-Model ROC Curves (LinearSVC AUC = 0.983)

4.3 Misclassification & Error Analysis

- **False Positives (Real news predicted as Fake):** Articles containing satire, quoted opinion pieces, or editorial columns occasionally share lexical patterns with sensationalist writing.
- **False Negatives (Fake news predicted as Real):** Elaborate fake news articles that mimic formal AP/Reuters journalistic style with neutral wording are more challenging to differentiate purely on lexical unigrams, benefiting from n-gram bigram features.

5. PRODUCTION CLOUD DEPLOYMENT & RAG DOCUMENT PIPELINE

Beyond local evaluation, the news classification system was containerized and deployed into production on **Vercel Serverless Cloud**. It features a custom **White Theme Web Application** with real-time inference and a dedicated **Retrieval-Augmented (RAG) Document Pipeline** for verifying multi-page PDF documents.

> **Live Production Web Deployment: https://fakenewsclassification.vercel.app**
> Features dual-mode real-time article verification, multi-page PDF document fact-checking, Explainable AI trigger keywords, and sub-15ms serverless inference.

5.1 Production Architecture & Cloud APIs

Production Cloud Host	Vercel Serverless Functions (Python 3.12 Runtime)
Live Public URL	`https://fakenewsclassification.vercel.app`
Real-Time Predict API	`POST /api/predict` (Accepts title & content JSON payload)
PDF RAG Audit API	`POST /api/analyze-pdf` (Multi-page PDF parsing and chunk analysis)
Model & Vectorizer Weights	`best_model.joblib` (LinearSVC, 64 KB) + `tfidf_vectorizer.joblib` (306 KB)
Inference Latency	< 15 ms average serverless response time

5.2 RAG Document Ingestion & Claim-by-Claim Fact-Checking

The system integrates a **Retrieval-Augmented Claim Verification Pipeline** tailored for news editors and researchers:

- **Document Ingestion & Page Extraction:** High-throughput text parsing across multi-page PDFs using `pypdf`.
- **Semantic Claim Chunking:** The document is partitioned into coherent informational assertion blocks (20–50 words), filtering noise, headers, and boilerplate.
- **Factual Integrity Audit:** Each claim is evaluated against the trained decision boundary, producing a granular per-page audit table with confidence metrics and influential trigger keywords.
- **Aggregated Authenticity Scoring:** The system produces an overall document health percentage and risk classification (**AUTHENTIC** vs. **SUSPICIOUS**).

5.3 Local Gradio Interface (Offline Standalone)

In addition to the Vercel web deployment, a local GUI is provided via `gradio_app.py` on port `7860` for offline desktop environments.

6. CONCLUSION & FINAL SUBMISSION DELIVERABLES

The project successfully meets all academic and industry requirements of **Project Statement 15**. The Linear Support Vector machine achieved state-of-the-art text classification performance with **93.69% accuracy**, an **F1-Score of 0.9367**, and an **ROC-AUC of 0.9833** on unseen holdout test articles.

DELIVERABLE ITEM	FILE NAME / URL	FORMAT	DESCRIPTION
Live Web Application	`https://fakenewsclassification.vercel.app`	URL	Public production web interface on Vercel Cloud
PDF Project Report	`Fake_News_Classification_Project_Report.pdf`	`.PDF`	Official publication-grade project documentation
Jupyter Notebook	`fake_news_detector.ipynb`	`.IPYNB`	End-to-end Python code, visualizations, and markdown commentary
Trained Model File	`best_model.joblib`	`.JOBLIB`	Serialized LinearSVC model for production inference
TF-IDF Vectorizer File	`tfidf_vectorizer.joblib`	`.JOBLIB`	Serialized 8,000 n-gram TF-IDF vocabulary matrix
Cloud Web App Script	`api/index.py`	`.PY`	Flask serverless application with White UI & RAG engine
Local GUI Script	`gradio_app.py`	`.PY`	Gradio web interface for local article verification
Complete Archive	`fake_news_classification_project.zip`	`.ZIP`	All project files packaged under 150 MB portal limit